strax + amstrax: the XAMS reconstruction stream

Explainer — from redax chunks to event_info, and where every plugin enters (amstrax docs/amstrax-overview-note b03534e 2026-10-18, generated from the source; 2026-10-18)

In one sentence. strax is a generic engine: it knows how to chain plugins, stream data in time chunks and store results under a hash of everything that made them. **amstrax** is the XAMS part: the plugins (what happens to pulses, peaks and events), the contexts (which plugins, where data lives, the run database) and the settings (run doc, corrections from amstrax_files). You ask for one data type; strax works out and runs everything below it.

1. strax in seven words

word	meaning	in amstrax
plugin	Python class: **depends_on** (inputs), **provides** (output name), **dtype** (columns), **compute()** (the work), options as class attributes	22 plugin classes in amstrax/plugins/
data type	named product, a numpy structured array; every row has time and endtime (ns since 1970)	raw_records, records, peaks, peak_basics, events, event_info, …
data kind	data types with the same rows (one per pulse / peak / event); a plugin that depends on several of the same kind gets them merged column-wise	'peaks' = peaks + peak_basics + peak_positions
context	registry of plugins + settings + storage list; st.get_array / st.make go through it	contexts.py:68 xams(), :119 context_for_daq_reader(), :216 xams_led()
option	setting of a plugin (default in the class, overridden with st.set_config); tracked options enter the lineage	XAMSConfig (xams_config.py:34): also file://, rundoc:// values
lineage	hash of plugin name + version + tracked options + lineage of the inputs: same hash = same data, change anything → new hash → recomputed, never mixed	data folders --
storage	where strax looks for / writes data, in order; save_when decides what is written	RunDB (rundb.py:141, run doc data[]) + DataDirectory folders

2. What happens when you ask for a data type

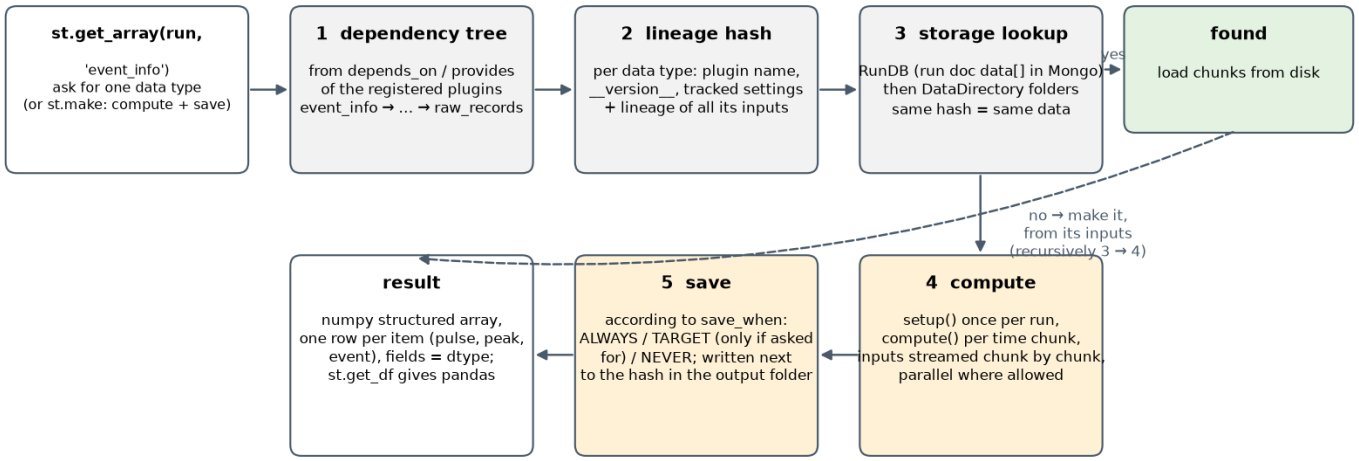


Figure 1. One request, generic strax. The dashed arrow: after computing (and saving) you get the same array as when it was loaded. Only what is missing is computed; a changed setting or plugin version makes a new lineage, so the old data is not used for it.

3. How the central processing on STBC runs it

auto_processing_new/process.py (main :538) runs per run, as xamsdata. **Stage 1** (:354): **raw_st.make(run, 'raw_records')** in the DAQ-reader context — only there may the DAQReader run; it reads the live data. **Stage 2** (:454): **st.make(run, target)** for the other targets in the main context; raw_records are only read there (forbid_creation_of). Targets come from the command line; peak_basics_ext is added when the run has the NaI channel. **Corrections**: --corrections_version (:517, default None) picks amstrax_files/corrections/_global/_global_vN.json; apply_global_correction_version (contexts.py:233) turns every entry into st.set_config. Without it, gain_to_pe_array stays None → to_pe = 1 → peak areas in ADC·samples, not PE. Since 2026-10-18 the online loop runs with **PROD0** (approved gains + position map): runs with approved gains get PE; later runs (null gains) stay in ADC·samples. Positions come from the map in both cases.

4. The reconstruction stream

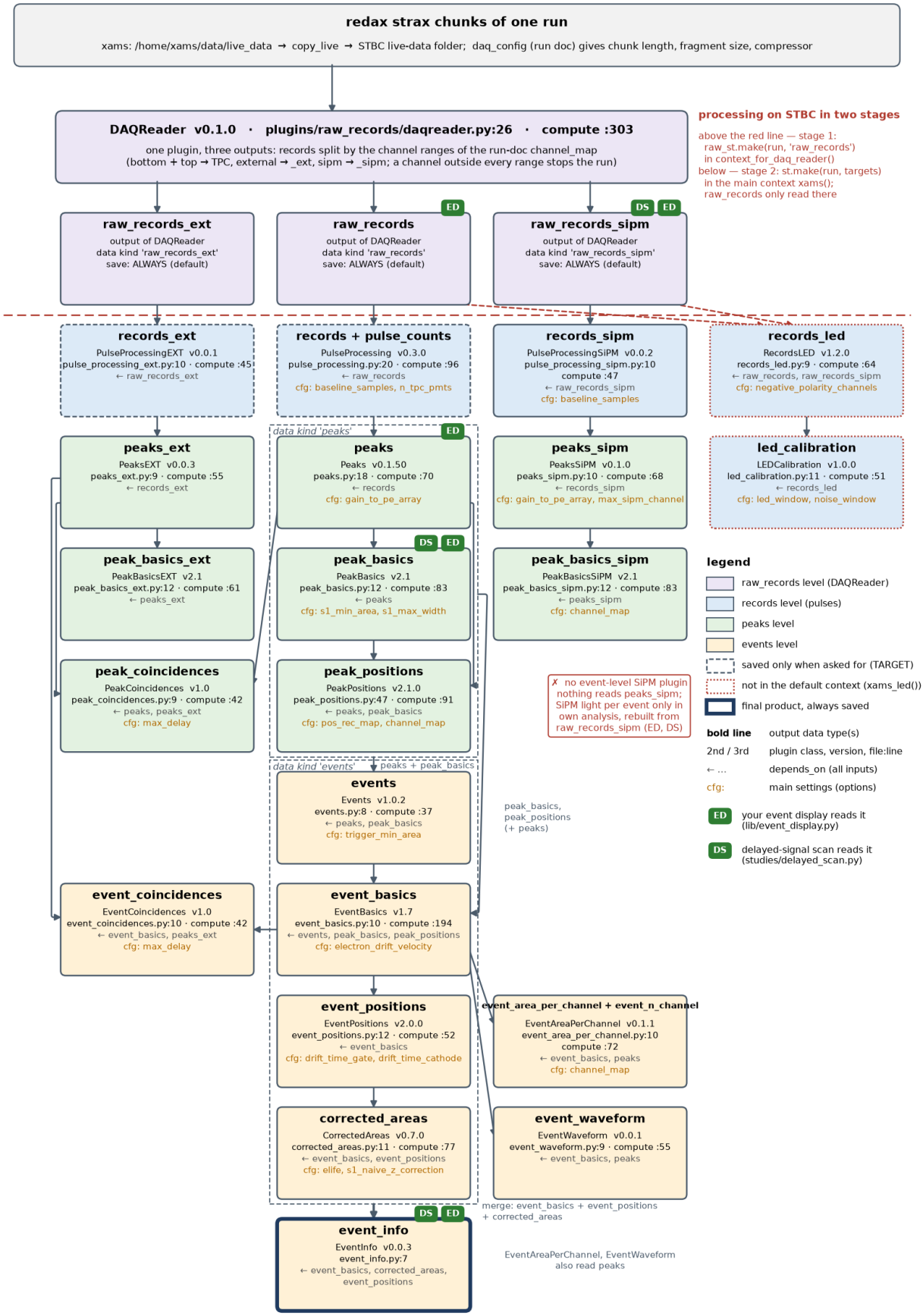


Figure 2. Every amstrax plugin and where it enters. Columns: external PMT (NaI) | TPC PMTs (main chain) | SiPMs | LED calibration. Each box is one plugin; its text is read from the source. Arrows show the main flow; the full input list is the ← line in each box. Dashed frames: one data kind (same rows, more columns). Registered in the default context: contexts.py:25.

5. Main chain, stage by stage

Paths relative to amstrax/amstrax/. The line numbers are those of the checkout named on page 1.

#	output (save)	plugin · file:line	depends_on	main settings (default)	what it does
1	**raw_records_sipm + raw_records_ext + raw_records** ALWAYS (default)	**DAQReader** v0.1.0 plugins/raw_records/daqreader.py:26 compute :303	(files)	channel_map = run doc daq_chunk_duration = int(5000000000.0)	Reads the redax chunk files of one run, splits the records by channel range (run-doc channel_map) into TPC / external / SiPM and makes the time absolute.
2	**records + pulse_counts** records: TARGET, pulse_counts: ALWAYS	**PulseProcessing** v0.3.0 plugins/records/pulse_processing.py:20 compute :96	raw_records	baseline_samples = 20 n_tpc_pmts = None	Keeps TPC channels; raw_to_records; baseline per channel from the first baseline_samples, flips the negative PMT pulses; integrates each pulse; counts pulses per channel.
3	**peaks** ALWAYS (default)	**Peaks** v0.1.50 plugins/peaks/peaks.py:18 compute :70	records	gain_to_pe_array = None peak_gap_threshold = 300 peak_min_pmts = 1 peak_min_area = 10	find_hits (samples above threshold) → find_peaks (hits closer than peak_gap_threshold, ≥ peak_min_pmts, area × to_pe) → sum_waveform → split_peaks → widths.
4	**peak_basics** ALWAYS (default)	**PeakBasics** v2.1 plugins/peaks/peak_basics.py:12 compute :83	peaks	s1_min_area = 10 s1_max_width = 225 s2_min_area = 10 s2_min_width = 225 channel_map = run doc	Area, widths, area_fraction_top, n_channels, saturation; type 1 = S1, 2 = S2, 0 = unknown from the width / area / AFT / channel cuts.
5	**peak_positions** ALWAYS (default)	**PeakPositions** v2.1.0 plugins/peaks/peak_positions.py:47 compute :91	peaks, peak_basics	pos_rec_map = DEFAULT_POS_REC_MAP channel_map = run doc	u, v from the top-array light fractions; x, y, r from pos_rec_map (default: latest map), with or without gains (without: ~2 mm worse); NaN without a map. No polynomial since 2.0.0.
6	**events** ALWAYS (default)	**Events** v1.0.2 plugins/events/events.py:8 compute :37	peaks, peak_basics	trigger_min_area = 10 left_event_extension = int(500000.0) right_event_extension = int(50000.0)	Triggers = peaks with area > trigger_min_area; window [t − left_event_extension, t + right_event_extension]; overlapping windows merged (find_peak_groups). Only time windows, no physics.
7	**event_basics** ALWAYS (default)	**EventBasics** v1.7 plugins/events/event_basics.py:10 compute :194	events, peak_basics, peak_positions	electron_drift_velocity = 1.6e-06 max_drift_length = amstrax.tpc_z	Per event: main and alt S1 / S2 (largest; S2 within the maximum drift), drift time, properties of those peaks (incl. s2_x, s2_y from peak_positions) copied into the event row.
8	**event_positions** ALWAYS (default)	**EventPositions** v2.0.0 plugins/events/event_positions.py:12 compute :52	event_basics	drift_time_gate = 3000 drift_time_cathode = 39500	x, y, r, phi of main and alt S2 from event_basics (map positions), u, v, z from the drift time (drift_time_gate, drift_time_cathode, gate_cathode_distance).
9	**corrected_areas** ALWAYS (default)	**CorrectedAreas** v0.7.0 plugins/events/corrected_areas.py:11 compute :77	event_basics, event_positions	elife = 30000 s1_naive_z_correction = [-52, 0, 700, -7.69]	cS1 (naive z correction) and cS2 (electron lifetime elife), each also split top / bottom.
10	**event_info** ALWAYS	**EventInfo** v0.0.3 plugins/events/event_info.py:7	event_basics, corrected_areas, event_positions	–	Merge only: event_basics + corrected_areas + event_positions in one table. What the analysis reads.

6. Side branches

#	output (save)	plugin · file:line	depends_on	main settings (default)	what it does
1	**records_ext** TARGET	**PulseProcessingEXT** v0.0.1 plugins/records_ext/pulse_processing_ext.py:10 compute :45	raw_records_ext	–	Baseline + integration for the external channel (NaI).

#	output (save)	plugin · file:line	depends_on	main settings (default)	what it does
2	**peaks_ext** ALWAYS (default)	**PeaksEXT** v0.0.3 plugins/peaks_ext/peaks_ext.py:9 compute :55	records_ext	–	Hit and peak finding on records_ext.
3	**peak_basics_ext** ALWAYS (default)	**PeakBasicsEXT** v2.1 plugins/peaks_ext/peak_basics_ext.py:12 compute :61	peaks_ext	–	Peak properties of the external peaks.
4	**peak_coincidences** ALWAYS (default)	**PeakCoincidences** v1.0 plugins/peaks/peak_coincidences.py:9 compute :42	peaks, peaks_ext	max_delay = 100	TPC peaks matched in time to external peaks (max_delay; 511-keV window).
5	**event_coincidences** ALWAYS (default)	**EventCoincidences** v1.0 plugins/events/event_coincidences.py:10 compute :42	event_basics, peaks_ext	max_delay = 100	Events matched in time to external peaks.
6	**records_sipm** ALWAYS (default)	**PulseProcessingSiPM** v0.0.2 plugins/records_ext/pulse_processing_sipm.py:10 compute :47	raw_records_sipm	baseline_samples = 20	Baseline + integration for the SiPM channels.
7	**peaks_sipm** ALWAYS (default)	**PeaksSiPM** v0.1.0 plugins/peaks_ext/peaks_sipm.py:10 compute :68	records_sipm	gain_to_pe_array = None max_sipm_channel = 7 channel_map = run doc	Hit and peak finding on records_sipm (area_per_channel up to max_sipm_channel).
8	**peak_basics_sipm** ALWAYS (default)	**PeakBasicsSiPM** v2.1 plugins/peaks_ext/peak_basics_sipm.py:12 compute :83	peaks_sipm	channel_map = run doc	Peak properties of the SiPM peaks.
9	**event_area_per_channel + event_n_channel** event_area_per_channel: ALWAYS, event_n_channel: ALWAYS	**EventAreaPerChannel** v0.1.1 plugins/events/event_area_per_channel.py:10 compute :72	event_basics, peaks	channel_map = run doc	Area per channel and number of channels of main / alt S1 and S2.
10	**event_waveform** EXPLICIT	**EventWaveform** v0.0.1 plugins/events/event_waveform.py:9 compute :55	event_basics, peaks	–	Summed waveform (total and top) of main / alt S1 and S2.
11	**records_led** TARGET	**RecordsLED** v1.2.0 plugins/led_calibration/records_led.py:9 compute :64	raw_records, raw_records_sipm	negative_polarity_channels = (0, 1, 2, 3, 4, 5)	LED window cut from raw_records + raw_records_sipm; flips only negative_polarity_channels.
12	**led_calibration** ALWAYS (default)	**LEDCalibration** v1.0.0 plugins/led_calibration/led_calibration.py:11 compute :51	records_led	led_window = (80, 110) noise_window = (0, 10)	Amplitude and area per LED trigger (led_window vs noise_window) → input for the gain fits.

7. Where the settings come from (later overrides earlier)

#	source	what / where
1	plugin default	default= of each option in the plugin class (column 'main settings' above)
2	context	XAMS_COMMON_CONFIG in contexts.py (now only n_tpc_pmts = 5); without a run database (init_rundb=False) also the default channel_map
3	run doc	options with a rundoc:// default read the run's own value: channel_map from xams_bookkeeping.channel_map, else daq_config.channel_map, else the XAMS default (xams_config.py:119). Plugins: DAQReader, EventAreaPerChannel, PeakBasics, PeakBasicsSiPM, PeakPositions, PeaksSiPM. DAQReader also gets chunk length, fragment size, compressor and threads from daq_config.
4	corrections_version	_global_vN.json in amstrax_files: file:// entries are fetched per run (run ranges inside the file), numbers set directly. Table below: v4.
5	st.set_config(…)	your own overrides in an analysis, e.g. PHOTON_CONFIG in studies/delayed_scan.py (50 ns gap, no minimum area) → own lineage, own data folder

setting	value in _global_v4.json	used by
elife	elife_v2.json	CorrectedAreas
gain_to_pe_array	gain_to_pe_array_v2.json	Peaks, PeaksEXT, PeaksSiPM
pos_rec_params	pos_rec_params_v0.json	*no plugin option of this name*
pos_rec_map	pos_rec_map_v0.json	PeakPositions
drift_time_gate	drift_time_gate_v0.json	EventPositions
drift_time_cathode	drift_time_cathode_v0.json	EventPositions
gate_cathode_distance	51.8	EventPositions
s1_naive_z_correction	[-51.8, 0.0, 700.0, -13.28]	CorrectedAreas
s1_min_area	10	PeakBasics, PeakBasicsSiPM
s1_min_width	10	PeakBasics, PeakBasicsEXT, PeakBasicsSiPM
s1_max_width	100	PeakBasics, PeakBasicsEXT, PeakBasicsSiPM
s2_min_area	100	PeakBasics, PeakBasicsSiPM
s2_min_width	150	PeakBasics, PeakBasicsEXT, PeakBasicsSiPM
s1_max_area_fraction_top	1	PeakBasics, PeakBasicsSiPM
s1_min_channels	1	PeakBasics, PeakBasicsSiPM
s2_min_channels	1	PeakBasics, PeakBasicsSiPM
left_event_extension	50000	Events
right_event_extension	50000	Events
max_drift_length	52	EventBasics
electron_drift_velocity	0.00104	EventBasics

8. Where the SiPM data is

Up to peaks only. raw_records_sipm → records_sipm → peaks_sipm → peak_basics_sipm. No plugin reads peaks_sipm, so nothing SiPM reaches events or event_info. The SiPM light per event (event display) and the SiPM pulse rate after an S2 (delayed scan) are rebuilt in your own analysis from raw_records_sipm, matched in time to event_info / peak_basics. Since PR #340 (2026-10-17) the plugins read the run's own channel_map and PeaksSiPM keeps both channels (6 and 7); before that, the context forced sipm = (6, 6) for every plugin except the DAQReader.

9. Where an analysis enters (example: the SR-2026.2 analysis, acolijn/xams-analysis-sr2026.2)

code	reads	own reconstruction
lib/event_display.py (ED)	event_info; peaks, peak_basics in the event window; raw_records, raw_records_sipm; to_pe of Peaks, baseline_samples of PulseProcessing	raw pulses per channel, own baseline, summed waveforms, SiPM PE in the S1 / S2 windows
studies/delayed_scan.py (DS)	peak_basics (central v4, and photon-mode in /tmp); event_info; raw_records_sipm; to_pe of Peaks	parents = merged large S2s; photon peaks and SiPM pulses attached to the last parent (time since its end)
scratch/sipm_after_s2.py	event_info, peak_basics, raw_records_sipm	cumulative SiPM PE in windows after the S2, minus the rate before the S1

10. Things to know

• **Positions: one algorithm**: the (u, v) → (x, y) map (posrec v2) per peak, by default the latest map (pos_rec_map_v1), with or without gains (without gains (u, v) are in ADC·samples: ~2 mm median shift); NaN only for runs without a map (before 7601). The old polynomial (pos_rec_params, x_corr) is gone. Production (PROD0) and analysis (v4, gain_to_pe_array_v2.json) give identical x, y for 7601–7723.
• **Event window**: the plugin defaults are left 500 µs / right 50 µs (Events); v4 sets both to 50 µs. Delayed light up to ms after an S2 is outside every event — so it is analysed on peaks, not events.
• **Polarity**: PulseProcessing flips every TPC channel (baseline_per_channel(flip=True)); the polarity registers of the run are not read. daq_config.extract_channel_polarity exists but nothing calls it now.
• **records are not kept**: PulseProcessing saves records only as a target (pulse_counts always). Anything above records needs the raw_records again if it must be recomputed.
• **Renewing**: python docs/notes/make_amstrax_note.py after a change to amstrax — names, versions, lines and dependencies follow; a new or removed plugin stops the script until it gets a place and a description.